Python Lab 1

Webserver Programming

Bikram Lamsal

School of Computer and Information Sciences, University of the Cumberlands

Advanced Computer Networks (MSCS-631-B01)

Dr. Dax Bradley

July 5, 2026

Introduction

Socket programming is an important foundation for understanding how application-layer services communicate across a network. In this lab, a simple Python-based TCP web server was developed to receive an HTTP request from a client browser, locate the requested HTML file on the server, and return the file contents in an HTTP response. The assignment instructions state that the server should accept one HTTP request at a time, parse the request, retrieve the requested file, send the file with HTTP response headers, and return a 404 Not Found response when the file is unavailable (University of the Cumberlands, n.d.). This exercise connects directly to the client-server model and HTTP request-response behavior described in computer networking coursework (Kurose & Ross, 2021).

Objective

The objective of this lab was to complete a skeleton Python web server using TCP sockets. The completed server needed to create a socket, bind it to a specific address and port, listen for incoming browser requests, receive and parse an HTTP request packet, send a valid HTTP response, and close the client connection after the response was delivered. The lab also required testing the server via a web browser by requesting both an existing HTML file and a missing one.

Methodology

The server was implemented using Python's socket module. A TCP socket was created using AF_INET and SOCK_STREAM, then bound to port 6789. After binding, the server listened for incoming connections and accepted one client request at a time. When a browser requested HelloWorld.html, the server extracted the requested filename from the HTTP GET

request, opened the file from the same directory as the server, and sent a 200 OK response with a text/html content type before sending the HTML body.

The program's error-handling section used an IOError exception to detect when the requested file did not exist. In that case, the server sent an HTTP/1.1 404 Not Found response and returned a short HTML message to the browser. This method demonstrated the difference between a successful HTTP response and an error response. Testing was performed using localhost with the URL http://localhost:6789/HelloWorld.html for the successful request and http://localhost:6789/anotherFile.html for the missing-file request.

Completed Server Code

```
"""Simple TCP web server for the socket programming lab."""
from __future__ import annotations

import argparse
import mimetypes
import socket
from pathlib import Path
from urllib.parse import unquote, urlparse


HOST = "127.0.0.1"
PORT = 6789
BUFFER_SIZE = 4096
WEB_ROOT = Path(__file__).resolve().parent


def build_response(status: str, body: bytes, content_type: str) -> bytes:
    """Build a minimal HTTP/1.1 response packet."""
    headers = [
        f"HTTP/1.1 {status}",
        f"Content-Length: {len(body)}",
        f"Content-Type: {content_type}",
        "Connection: close",
```

```
        "",
        "",
    ]
    return "\r\n".join(headers).encode("iso-8859-1") + body


def requested_file(path: str) -> Path:
    """Resolve the requested HTTP path inside the lab directory."""
    parsed_path = unquote(urlparse(path).path)
    if parsed_path == "/":
        parsed_path = "/HelloWorld.html"

    relative_path = parsed_path.lstrip("/")
    candidate = (WEB_ROOT / relative_path).resolve()
    if WEB_ROOT not in candidate.parents and candidate != WEB_ROOT:
        raise PermissionError("Requested path is outside the web root.")
    return candidate


def handle_request(message: str) -> bytes:
    request_line = message.splitlines()[0] if message.splitlines() else ""
    parts = request_line.split()

    if len(parts) < 3:
        body = b"<html><body><h1>400 Bad Request</h1></body></html>"
        return build_response("400 Bad Request", body, "text/html; charset=utf-8")

    method, path, _version = parts
    if method != "GET":
        body = b"<html><body><h1>405 Method Not Allowed</h1></body></html>"
        return build_response("405 Method Not Allowed", body, "text/html; charset=utf-8")

    try:
        filename = requested_file(path)
        body = filename.read_bytes()
        content_type = mimetypes.guess_type(filename.name)[0] or "application/octet-stream"
        return build_response("200 OK", body, content_type)
```

```
    except (FileNotFoundError, IsADirectoryError):
        body = (
            b"<html><head><title>404 Not Found</title></head>"
            b"<body><h1>404 Not Found</h1><p>The requested file was not found.</p></body></html>"
        )
        return build_response("404 Not Found", body, "text/html; charset=utf-8")
    except PermissionError:
        body = b"<html><body><h1>403 Forbidden</h1></body></html>"
        return build_response("403 Forbidden", body, "text/html; charset=utf-8")


def run_server(host: str, port: int) -> None:
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server_socket.bind((host, port))
    server_socket.listen(1)

    print(f"Ready to serve on http://{host}:{port}/HelloWorld.html")

    try:
        while True:
            connection_socket, address = server_socket.accept()
            with connection_socket:
                print(f"Accepted connection from {address[0]}:{address[1]}")
                message = connection_socket.recv(BUFFER_SIZE).decode("iso-8859-1")
                response = handle_request(message)
                connection_socket.sendall(response)
    except KeyboardInterrupt:
        print("\nServer stopped.")
    finally:
        server_socket.close()


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Run the Python socket web server lab.")
    parser.add_argument("--host", default=HOST, help=f"Host address to bind. Default: {HOST}")
    parser.add_argument("--port", type=int, default=PORT, help=f"Port to bind. Default: {PORT}")
```

```
    return parser.parse_args()


if __name__ == "__main__":
    args = parse_args()
    run_server(args.host, args.port)
```

Sample HTML Test File

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Python Socket Web Server</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 40px;
            line-height: 1.5;
            color: #1f2933;
        }

        main {
            max-width: 760px;
        }

        h1 {
            color: #0b5cab;
        }
    </style>
</head>
<body>
    <main>
        <h1>Hello from my Python TCP Web Server</h1>
        <p>
            This page was delivered by a socket program that accepts a TCP
```

```
        connection, reads an HTTP GET request, and sends back an HTTP
        response containing this HTML file.
      </p>
      <p>
        The server is running locally on port 6789 for Python Lab 1.
      </p>
    </main>
</body>
</html>
```

Testing and Results

The server was tested by placing HelloWorld.html in the same directory as WebServer.py and running the Python server. When the browser requested the existing HTML file, the server returned the correct page content with an HTTP 200 OK response. This confirmed that the server socket accepted the connection, read the HTTP request, located the file, and transmitted the HTML content to the client browser.

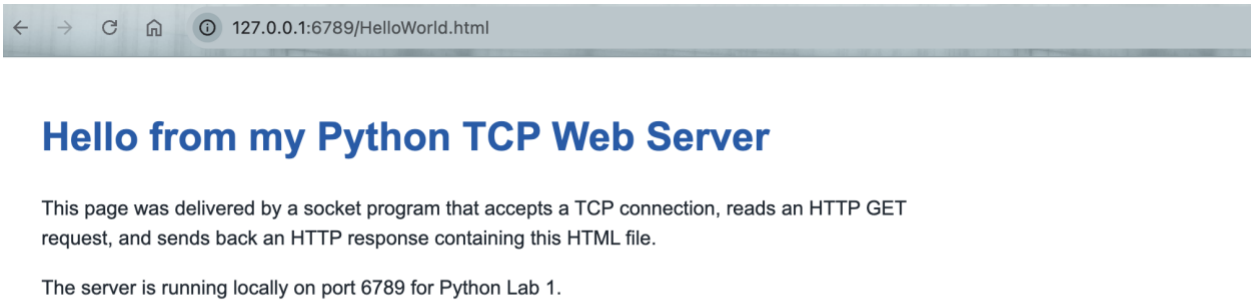

Figure 1: Successful browser response for HelloWorld.html.

The second test used a URL for a file that was not present in the server directory. The browser displayed a 404 Not Found message, confirming that the program's exception-handling

section worked correctly. This result showed that the server could respond appropriately when a requested resource was unavailable.

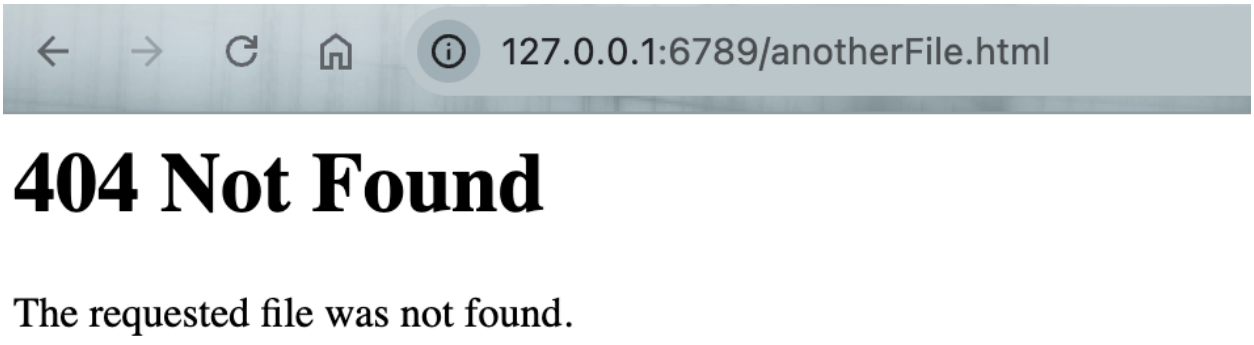

Figure 2: Browser response for a missing file request

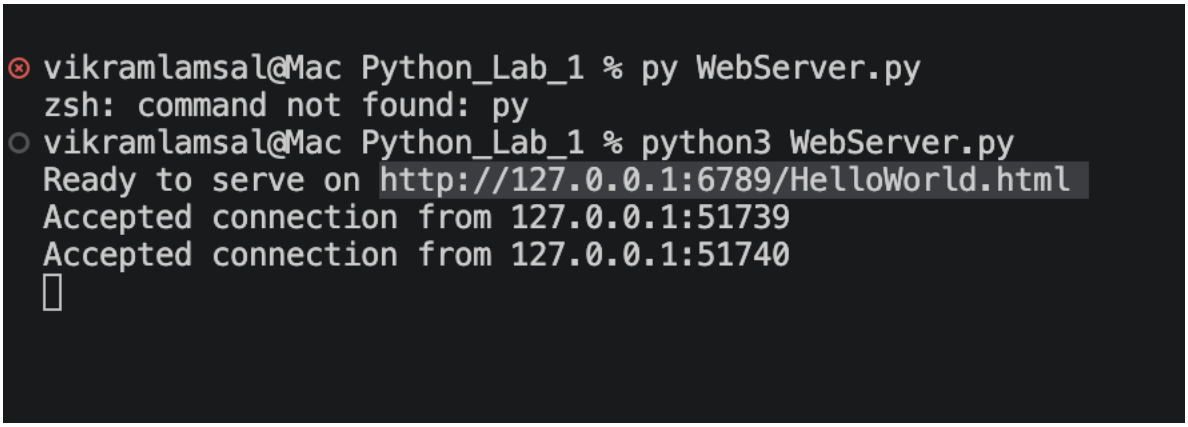

Figure 3: Terminal running WebServer successfully

Experience and Challenges

This lab helped me better understand the relationship between TCP sockets and HTTP communication. Before completing the lab, I understood that a browser sends requests to a server, but building the server manually made the process much clearer. I learned that the

browser request includes the filename, and the server must parse the request, locate the file, create a response header, and send the file content back over the established TCP connection. Completing the skeleton code also helped me see how lower-level socket programming supports higher-level web communication.

One of the main challenges was understanding the exact format required for HTTP headers. The response needed a status line, a content-type header, and a blank line before the HTML content. Another challenge was ensuring the requested filename was correctly extracted from the browser request and that the HTML file was stored in the same directory as the Python server. The 404 test was also useful because it showed how a server should handle unavailable resources instead of simply failing. Overall, the lab provided practical experience with TCP connections, HTTP response formatting, and basic error handling in a network application.

Conclusion

The completed web server successfully demonstrated the basic operation of a TCP-based HTTP server. It accepted client connections, received browser requests, retrieved an HTML file, returned it with a proper HTTP response, and handled missing files with a 404 Not Found response. Although the server is simple and handles only one request at a time, it provides a strong foundation for understanding how real web servers communicate with browsers using TCP and HTTP.

References

Kurose, J. F., & Ross, K. W. (2021). Computer networking: A top-down approach (8th ed.). Pearson.

University of the Cumberlands. (n.d.). Lab 1: Web server lab [Course handout]. MSCS-631-B01 Advanced Computer Networks.